

Testing Guide

Ghana News Aggregator Testing Documentation

Version 2.0 | December 20, 2024

Table of Contents

- [1. Overview](#)
 - [2. Testing Strategy](#)
 - [3. Unit Testing](#)
 - [4. Integration Testing](#)
 - [5. End-to-End Testing](#)
 - [6. Interactive Testing](#)
 - [7. Accessibility Testing](#)
 - [8. Performance Testing](#)
 - [9. Security Testing](#)
 - [10. Continuous Integration](#)
-

Overview

This document provides comprehensive guidance on testing the Ghana News Aggregator system. Our testing strategy ensures high quality, reliability, and accessibility across all components.

Testing Objectives

- ✓ Ensure all 8 news sources are properly integrated
- ✓ Verify dashboard displays accurate real-time data
- ✓ Validate admin authentication and authorization

- ☒ Confirm accessibility compliance (WCAG 2.1 Level AA)
 - ☒ Test theme switching functionality
 - ☒ Verify audit logging completeness
 - ☒ Ensure performance meets SLA requirements
-

Testing Strategy

Test Pyramid



Test Coverage Goals

- **Unit Tests:** $\geq 80\%$ code coverage
 - **Integration Tests:** All API endpoints
 - **E2E Tests:** 7 critical user journeys
 - **Accessibility:** WCAG 2.1 Level AA compliance
 - **Performance:** Response time $< 500\text{ms}$ (p95)
-

Unit Testing

Setup

```
bash
```

```
# Install dependencies
```

```
cd backend
```

```
npm install
```

```
# Run unit tests
```

```
npm test
```

```
# Run with coverage
```

```
npm run test:coverage
```

Test Structure

```
javascript
```

```
describe('ArticleService', () => {  
  describe('fetchArticles', () => {  
    it('should fetch articles from all enabled sources', async () => {  
      // Arrange  
      const mockSources = [  
        { id: 'src-001', name: 'GhanaWeb', enabled: true },  
        { id: 'src-002', name: 'Graphic Online', enabled: true }  
      ];  
  
      // Act  
      const result = await articleService.fetchArticles(mockSources);  
  
      // Assert  
      expect(result).toHaveLength(2);  
      expect(result[0].source_id).toBe('src-001');  
    });  
  });  
});
```

Coverage Requirements

Component	Minimum Coverage
Services	85%
Controllers	80%
Utilities	90%
Models	75%

Integration Testing

API Testing

Test all REST endpoints with various scenarios:

```
bash

# Run integration tests
npm run test:integration
```

Test Cases

1. Dashboard Statistics

```
http

GET /api/stats/dashboard
Expected: {
  "totalArticles": number,
  "todayArticles": number,
  "activeSources": number,
  "totalSources": 8,
  "systemHealth": "healthy" | "degraded"
}
```

2. Sources Connectivity

http

GET /api/sources/connectivity

Expected: Array of 8 source objects with status

3. Admin Authentication

http

POST /api/admin/login

Body: {

 "username": "admin",

 "password": "admin123"

}

Expected: {

 "token": "jwt.token.here",

 "user": { ... }

}

4. Audit Logs

http

GET /api/admin/audit-logs?page=1&limit=50

Headers: Authorization: Bearer <token>

Expected: {

 "logs": [...],

 "pagination": { ... }

}

Database Integration

Test database operations:

javascript

```
describe('Database Operations', () => {
  it('should create article with all fields', async () => {
    const article = {
      id: 'test-001',
      title: 'Test Article',
      source_id: 'src-001',
      // ... other fields
    };

    await db.query('INSERT INTO articles ...', article);
    const [result] = await db.query('SELECT * FROM articles WHERE id = ?', [article.id]);

    expect(result[0].title).toBe('Test Article');
  });
});
```

End-to-End Testing

Puppeteer Test Suite

Our E2E tests use Puppeteer to simulate real user interactions with screenshot capture.

Running E2E Tests

```
bash

# Start the application
docker-compose up -d

# Wait for services to be ready
sleep 10

# Run E2E tests
cd tests/e2e
npm install
node puppeteer-tests.js
```

Test Scenarios

1. Dashboard Loading

- ☒ Navigate to homepage
- ☒ Verify dashboard header displays
- ☒ Check all 4 KPI cards load
- ☒ Validate KPI values are numbers
-  Screenshot: `dashboard-initial-load.png`

2. Feed Connectivity Grid

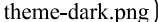
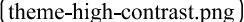
- ☒ Verify 8 source cards display
- ☒ Check each source has status badge
- ☒ Validate source statistics
- ☒ Verify color-coded status indicators
-  Screenshot: `feed-connectivity-grid.png`

3. Admin Authentication

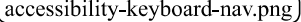
- ☒ Navigate to admin panel
- ☒ Fill login form
- ☒ Test password visibility toggle
- ☒ Submit credentials
- ☒ Verify successful login
-  Screenshot: `admin-panel-loaded.png`

4. Theme Switching

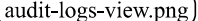
- ☒ Open theme selector
- ☒ Switch to Dark theme
- ☒ Switch to High Contrast theme

-  Verify CSS variables update
-  Test font size adjustment
-  Screenshots: , 

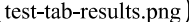
5. Accessibility Features

-  Test skip link (Tab key)
-  Verify ARIA labels present
-  Check keyboard navigation
-  Validate proper heading hierarchy
-  Test screen reader compatibility
-  Screenshot: 

6. Audit Log Viewer

-  Login as admin
-  Navigate to audit logs
-  Verify table displays
-  Check column headers
-  Validate log entries
-  Screenshot: 

7. Interactive Test Tab

-  Navigate to test tab
-  Click "Run Tests" button
-  Verify tests execute
-  Check test results display
-  Validate status indicators
-  Screenshot: 

Screenshot Capture

All screenshots are automatically saved to:

```
/tests/e2e/screenshots/
```

With naming format:

```
YYYY-MM-DDTHH-mm-ss_test-name.png
```

Test Reports

After test execution, view the HTML report:

```
/tests/e2e/screenshots/test-report.html
```

Example report includes:

-  Total tests run
-  Pass/fail counts
-  Success rate percentage
-  Individual test details with screenshots
-  Error messages for failures

Interactive Testing

Using the Test Tab

The application includes a built-in interactive testing interface:

1. Navigate to the "Tests" tab in the main navigation
2. Click "Run Tests" button
3. Watch real-time test execution

4. View results with status indicators

Test Categories

API Health Checks:

- Dashboard statistics endpoint
- Sources connectivity endpoint
- Category breakdown endpoint

Response Time Validation:

- Each test displays execution time
- Threshold: < 500ms

Status Verification:

- ✓ Success (HTTP 200-299)
 - ✗ Failure (HTTP 400-599)
 - ⚠ Error (network/timeout)
-

Accessibility Testing

WCAG 2.1 Compliance

Level AA Requirements

Perceivable

- Text alternatives for non-text content
- Adaptable content structure
- Distinguishable color contrast (4.5:1 minimum)

Operable

- Keyboard accessible
- No keyboard traps
- Skip navigation links
- Descriptive page titles

✓ Understandable

- Readable text (adjustable font size)
- Predictable navigation
- Clear error identification

✓ Robust

- Proper semantic HTML
- ARIA attributes where needed
- Compatible with assistive technologies

Testing Tools

Automated Testing:

```
bash

# Install axe-core
npm install @axe-core/puppeteer

# Run accessibility audit
node tests/accessibility-audit.js
```

Manual Testing:

```
bash
```

Screen reader testing

- NVDA (Windows)
- JAWS (Windows)
- VoiceOver (macOS/iOS)

Keyboard navigation

- Tab through all interactive elements
- Verify focus indicators visible
- Test skip links

Color contrast

- Use browser DevTools
- Chrome Lighthouse audit
- WebAIM Contrast Checker

Theme-Specific Testing

Test all three themes for accessibility:

1. Light Theme

- Background:
- Text:
- Contrast ratio: 21:1 

2. Dark Theme

- Background:
- Text:
- Contrast ratio: 15.8:1 

3. High Contrast

- Background:
 - Text:
 - Contrast ratio: 21:1 
 - Border: 2px solid white
-

Performance Testing

Load Testing

```
bash

# Install Artillery
npm install -g artillery

# Run load test
artillery run tests/performance/load-test.yml
```

Performance Benchmarks

Metric	Target	Critical
Dashboard Load	< 2s	< 3s
API Response (p50)	< 200ms	< 500ms
API Response (p95)	< 500ms	< 1000ms
API Response (p99)	< 1000ms	< 2000ms
Concurrent Users	1000	500

Test Scenarios

Scenario 1: Normal Load

- 100 concurrent users
- 10 requests per second
- Duration: 5 minutes

Scenario 2: Peak Load

- 500 concurrent users

- 50 requests per second
- Duration: 10 minutes

Scenario 3: Stress Test

- 1000+ concurrent users
- Increase until system degrades
- Identify breaking point

Monitoring

Track these metrics during tests:

- Response times (avg, p50, p95, p99)
 - Error rates
 - CPU usage
 - Memory usage
 - Database query times
 - Network bandwidth
-

Security Testing

Authentication Testing

Test Cases:

1.  Valid credentials → Success
2.  Invalid username → 401 Unauthorized
3.  Invalid password → 401 Unauthorized
4.  Missing credentials → 400 Bad Request
5.  SQL injection attempt → Sanitized
6.  XSS attempt → Escaped

7.  Expired token → 403 Forbidden
8.  Malformed token → 403 Forbidden

Authorization Testing

Test Cases:

1.  Admin routes without token → 401
2.  Admin routes with valid token → 200
3.  Source toggle requires admin → Protected
4.  Audit logs require admin → Protected

Input Validation

Test all endpoints for:

- SQL injection
- XSS attacks
- CSRF protection
- Request size limits
- Rate limiting

Audit Log Verification

Ensure all admin actions are logged:

```
sql
SELECT * FROM audit_logs
WHERE action IN ('LOGIN', 'ENABLE_SOURCE', 'DISABLE_SOURCE', 'MANUAL_REFRESH')
ORDER BY timestamp DESC;
```

Continuous Integration

GitHub Actions Workflow

yaml

name: CI/CD Pipeline

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **name:** Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '20'

- **name:** Install dependencies

run: npm ci

- **name:** Run unit tests

run: npm test

- **name:** Run E2E tests

run: |

docker-compose up -d

sleep 10

cd tests/e2e && npm install && node puppeteer-tests.js

- **name:** Upload test artifacts

uses: actions/upload-artifact@v3

with:

name: test-results

path: |

tests/e2e/screenshots/

coverage/

Pre-commit Hooks

```
bash

# Install Husky
npm install --save-dev husky

# Setup hooks
npx husky install

# Add pre-commit hook
npx husky add .husky/pre-commit "npm test"
```

Test Maintenance

Best Practices

1. Keep Tests Independent

- Each test should run in isolation
- No dependencies between tests
- Clean up after each test

2. Use Meaningful Names

```
javascript

// Bad
it('test 1', () => { ... });

// Good
it('should fetch articles when all 8 sources are enabled', () => { ... });
```

3. Follow AAA Pattern

- Arrange: Set up test data
- Act: Execute the code

- Assert: Verify the result

4. **Mock External Dependencies**

- Mock API calls
- Mock database queries
- Mock time-dependent functions

5. **Regular Test Reviews**

- Remove obsolete tests
- Update for new features
- Refactor duplicated logic

Troubleshooting

Common Issues:

1. **E2E Tests Timing Out**

- Increase wait timeouts
- Check if services are running
- Verify network connectivity

2. **Flaky Tests**

- Add explicit waits
- Use stable selectors
- Avoid race conditions

3. **Low Coverage**

- Identify uncovered code
 - Write missing tests
 - Remove dead code
-

Appendix

Test Data

Default Admin Credentials:

- Username:
- Password:
- ⚠ Change in production!

Mock Article Categories:

- Politics
- Business
- Sports
- Entertainment
- Technology
- Health

Expected Sources (8 total):

1. GhanaWeb
2. Graphic Online
3. MyJoyOnline
4. CitiNewsRoom
5. JoyNews
6. Pulse Ghana
7. Peace FM Online
8. Modern Ghana

Useful Commands

```
bash

# Run all tests
npm run test:all

# Run specific test file
npm test -- tests/services/article.test.js

# Run tests in watch mode
npm test -- --watch

# Generate coverage report
npm run test:coverage

# Run E2E tests with headed browser (for debugging)
HEADLESS=false node tests/e2e/puppeteer-tests.js

# Clean test artifacts
rm -rf tests/e2e/screenshots/* coverage/
```

Document Version: 2.0

Last Updated: December 20, 2024

Maintained By: Ghana News Aggregator Team